

Repository Management Plan

Version: 1.0

Date: 10/05/2025

Owner: [CV9 / Michał Bątkowski]

1. Branching Strategy: GitHub Flow

We will follow the GitHub Flow strategy for version control and collaboration.

Main principles:

- Work is done in feature branches
- Each feature branch is branched off main
- Once work is complete and tested, a Pull Request (PR) is made into main
- Code must be reviewed and approved before merging
- After the merger, branches are deleted

Naming Convention:

Type	Format	Example
Feature	feature/<short-description>	feature/add-logging
Bugfix	bugfix/<short-description>	bugfix/fix-inference-error
Hotfix	hotfix/<short-description>	hotfix/update-requirements
Docs	docs/<short-description>	docs/update-readme
Review	review/<short-description>	review/fix-on-repo-plan

Note: Branch names should be intuitive, easily understandable, and not too long. These conventions are flexible, use good judgment when naming.

2. Commit Message Guidelines

Use clear and consistent commit messages. Follow this structure:

<type>

[short summary, optional]

Types:

- feat: new feature
- fix: bug fix
- docs: documentation only
- refactor: code change that neither fixes a bug nor adds a feature
- test: adding or modifying tests
- chore: maintenance (e.g., formatting, build scripts)
- self study: self study evidence and materials
- any other: any that is outside of this scope

Examples:

- feat: add CLI interface for inference
- fix: correct path bug in data loader
- docs: update setup instructions in README

3. Pull Request (PR) Process

All code changes must go through a Pull Request.

PR Requirements:

- Title should be clear and follow the branch name
- Description must include:
 - Summary of changes
 - Related Azure DevOps task ID (if applicable)
 - Screenshots or logs if relevant
- Assign at least one reviewer
- Address all review comments before merging
- Do not self-approve your PR

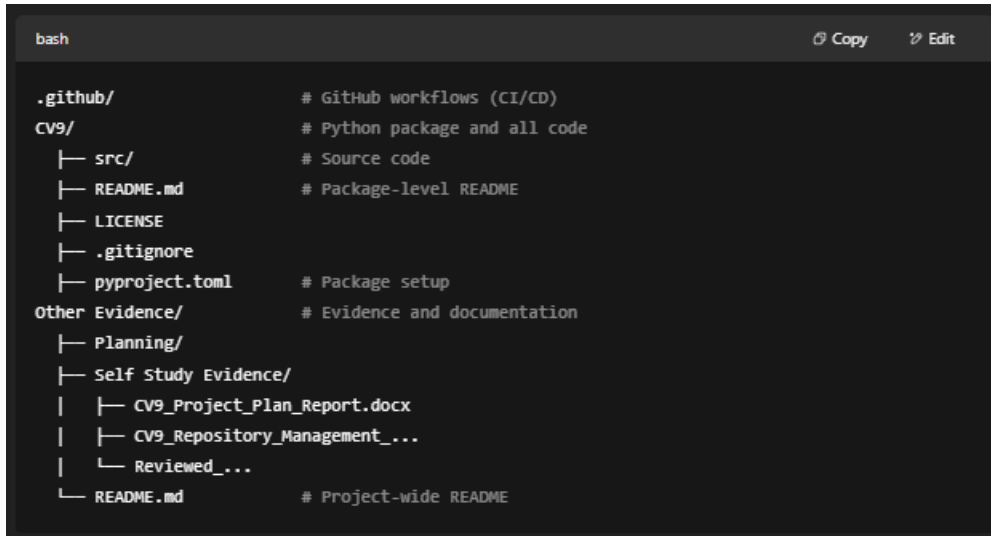
4. Protection Rules

main is a protected branch:

- Direct commits are not allowed
- PR approval is required before merging
- PRs must pass automated tests (if CI/CD is configured)

5. Repository Structure

Our repository is divided into two main areas to keep code and evidence cleanly separated:



```
bash                                                                    Copy Edit

.github/                        # Github workflows (CI/CD)
CV9/                            # Python package and all code
├─ src/                        # Source code
├─ README.md                  # Package-level README
├─ LICENSE
├─ .gitignore
├─ pyproject.toml             # Package setup
Other Evidence/                # Evidence and documentation
├─ Planning/
├─ Self Study Evidence/
│   ├── CV9_Project_Plan_Report.docx
│   ├── CV9_Repository_Management_...
│   └─ Reviewed_...
└─ README.md                  # Project-wide README
```

- **Folder Purpose:**
 - **CV9/** contains all source code, infrastructure, Python package, and setup files.
 - **Other Evidence/** stores deliverables like planning documents, reports, and study logs.
 - **.github/** is reserved for GitHub Actions and configuration files.

Keep work items in the correct folder. Code changes go under CV9/. Evidence files go under Other Evidence/.

!!!This is subject to change as we go further on into the project!!!

Acknowledgment

The structure and content of this repository management plan were partially assisted by ChatGPT to ensure clarity, consistency, and alignment with best practices in GitHub and MLOps documentation.